

Yazılım Süreçleri & Yazılım Geliştirme Modelleri



Yazılım Süreci

- Temel varsayımlar
 - İyi bir yazılım istiyorsak süreci iyi yönetmeliyiz
 - İyi bir süreç yönetimi riskleri azaltır
- Risk yönetimi
 - Bir yazılım projesinde neler yanlış gidebilir?
 - Risk nasıl azaltılabilir?

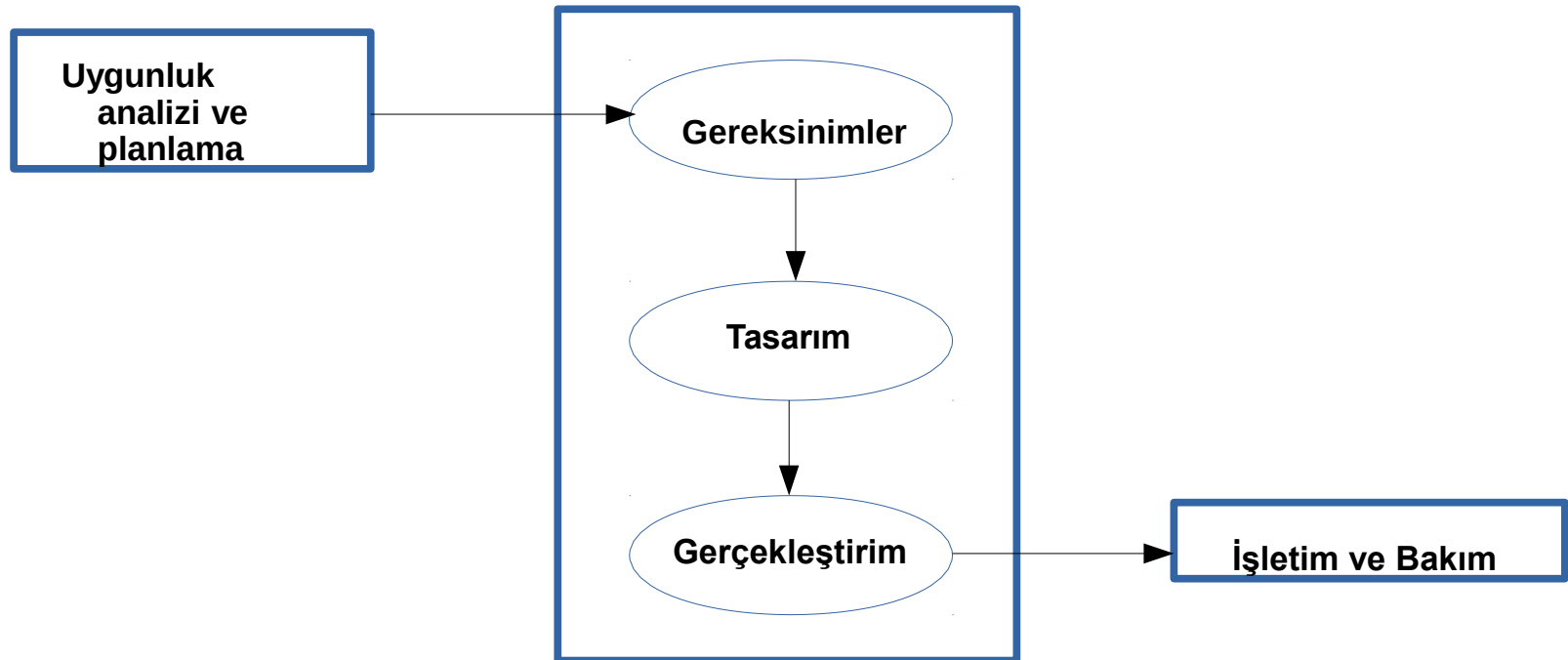
Yazılım Süreci

- ✂ Bir yazılım süreci, bir yazılım ürünü ortaya çıkarmak için yapılması gereken, birbiri ile ilişkili işlemler bütünüdür.
- ✂ Yazılım süreçleri farklılıklar gösterebilir ancak temel olarak dört işi içermelidir:
 - **Yazılımı tanımlama (software specification)** - Yazılımın ne yapacağı, kısıtları tanımlanmalıdır.
 - **Yazılım tasarımı ve gerçekleştirim (software design and implementation)** - Yazılım, tanımına uygun olarak üretilmelidir.
 - **Yazılımın doğrulanması (software validation)** - Yazılımın müşterinin isteklerini karşılayabildiği doğrulanmalıdır.
 - **Yazılımın evrimi (software evolution)** - Yazılım müşterinin değişen ihtiyaçlarına cevap verecek şekilde değiştirilebilmelidir.

Yazılım Süreci

- Süreci sadece veri modeli belirleme, kullanıcı arayüzü tasarımı gibi aktiviteler içermez
- Süreç şunları da içerebilir:
 - Ürünler
 - Roller
 - Ön ve son koşullar

Yazılım Süreci (basitleştirilmiş)



Yazılım Geliştirme Modelleri

- ☞ Kaos (Herhangi bir modelin uygulanmadığı o dönem)
- ☞ Şelale (WaterFall) yazılım geliştirme modeli
- ☞ Artırımsal (Incremental) ve tekrar işlemeli (İteratif) yazılım geliştirme modeli
- ☞ Yalın yazılım geliştirme modeli
- ☞ Re-use (tekrar kullanımlı) yazılım geliştirme modeli
- ☞ Prototip yazılım geliştirme modeli
- ☞ Test güdümlü yazılım geliştirme modeli
- ☞ Temiz oda yazılım geliştirme modeli
- ☞ Yazılım kalite unsurları

Yazılım Geliştirme Modelleri

∞ Kaos (Herhangi Bir Modelin Uygulanmadığı Dönem)

Herhangi bir metot veya model uygulanmayan zamanlardır. Kovboy kodlaması da denilir.

Aşağıda herhangi bir modelin uygulanmadığı sorunlu bir proje örneği verdik:

- Projede tutarlı dokümanlar hazırlanmamaktadır.
- Önce tüm yazılım kodlanır sonra hatalar düzeltilmeye çalışılır. Sistem giderek bir canavara benzer.
- Bakım maliyetleri her geçen gün artar, yeni yazılımcılar sisteme dokunmaya korkarlar.
- Kod taşınabilir değildir.
- Başarı bireylere bağlıdır, bireyler ayrılınca başarı dramatik biçimde düşer.
- Donanım seçimi iyi yapılmamıştır ve donanım ile yazılım iyi uyuşmamıştır, donanım genelde büyüyen yazılıma dar gelir.
- Yanlış seçilen algoritmalar vardır ve geri dönülememektedir.

Yazılım Geliştirme Modelleri

☞ Kaos (Herhangi Bir Modelin Uygulanmadığı Dönem) (devam)

- Yazılımda birkaç dil birden kullanılmıştır ve makine dili ile üretilen kod bölgelerini kimse anlamamaktadır.
- Yazılım kurnaz yazılım numaraları ile doludur ve kısa bir alanı anlamak saatleri ve hatta günleri almaktadır.
- Yazılım hala devam etmekteyse ne zaman biteceğini hiç kimse bilmemektedir ve projeye sürekli adam alınmaktadır veya adamlar ayrılmaktadır. Bu yüksek adam değişim oranı yüzünden projeye hakim pek fazla adam kalmamıştır.
- Donanımcılar yazılımcıları ve yazılımcılar donanımcıları suçlamaktadırlar.
- Sistemin bir bölümü çalışmakta bir bölümü çalışmamaktadır. Önemli bölümleri çalışmadığı halde ekip çok önemsiz ama müşterinin istediği bir bölümde zaman kaybetmektedir.
- Projenin iptali bile gündemdedir.

Yazılım Geliştirme Modelleri

∞ Şelale (WaterFall) Yazılım Geliştirme Modeli

- Şelale yazılım geliştirme modeli en klasik yazılım geliştirme modelidir.
- Bu modelde yazılım geliştirme işlemi kesin safhalara ayrılmıştır. Bu safhalar kabaca aşağıdaki gibidir:
 - Gereksinimlerin toplanması aşaması,
 - Tasarım aşaması,
 - Gerçekleştirme aşaması,
 - Doğrulama aşaması,
 - Bakım aşaması
- Genellikle yukarıdaki safhalar için önceden tahmini süreler belirlenir ve bu sürelerle sıkıca uyulmaya çalışılır.
- Şelale geliştirme modelinin en önemli özelliği safhaların ardarda uygulanması ve bir safha bitmeden bir sonraki safhaya geçilmemesidir.
- Şu an gerçekleştirilen safhadan önceki safhalara da bu safhalar tamamlanmış kabul edildiğinden normal koşullarda pek geri dönülmez.
- Her aşamada yoğun olarak dokümantasyon yazılır ve hatta bu dokümantasyonlar safha geçişlerini belirler.

Yazılım Geliştirme Modelleri

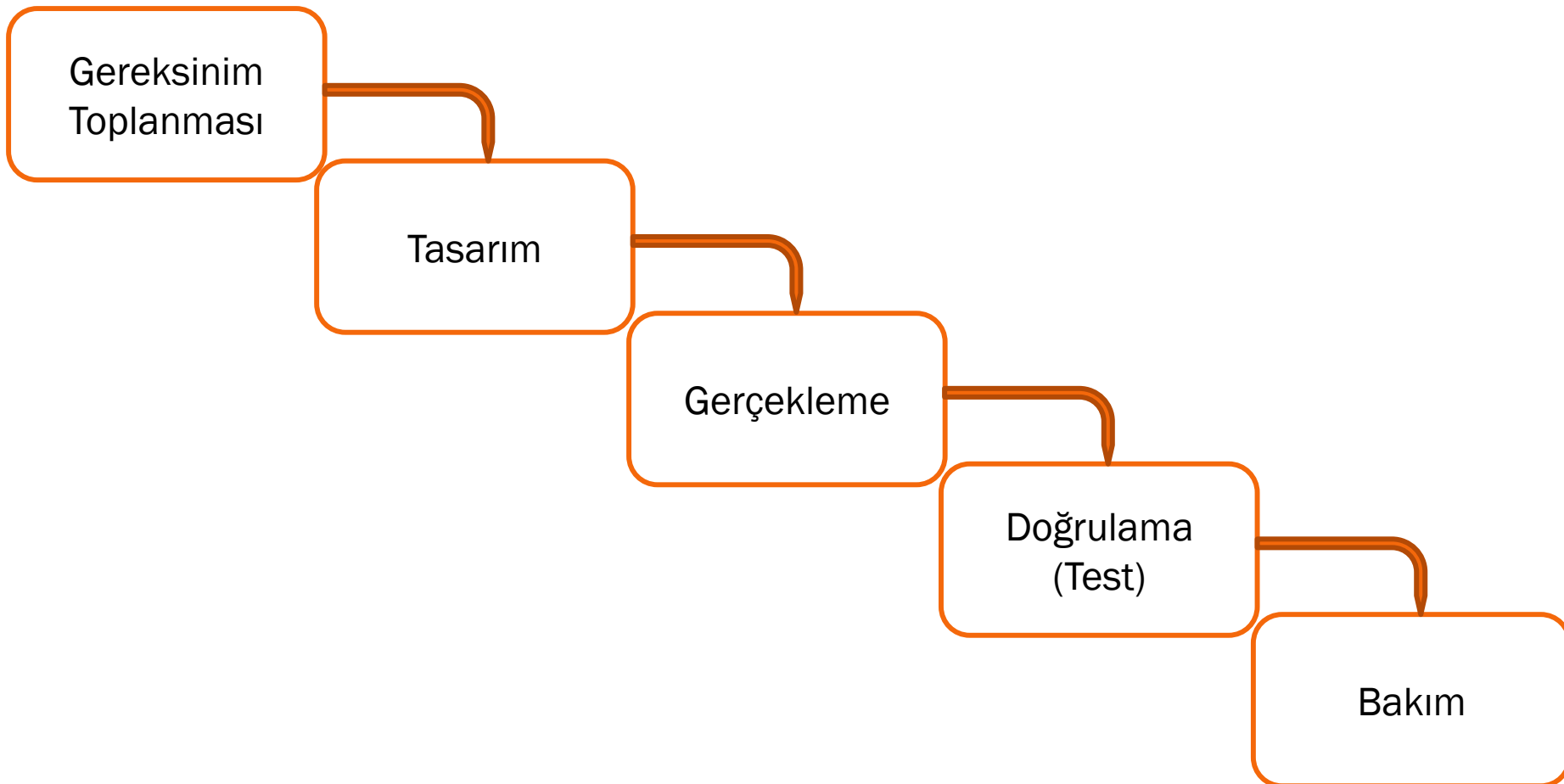
🌀 Şelale (WaterFall) Yazılım Geliştirme Modeli (devam)

Şelale geliştirme modelinin aşamalarını biraz daha detaylı inceleyelim:

- **Gereksinimlerin toplanması aşaması:** Bu aşamada problem tanımlanır veya kullanıcı isteği analiz edilir ve ilgili gereksinimler doküman olarak oluşturulur. Bu aşamada müşteri veya kullanıcı ile işbirliği yapılır ve bir sözleşme mahiyetindeki gereksinim ortaya çıkarılır. Bu gereksinim dokümanı diğer safhalar için bir ana kurallar dizisi olarak oluşur ve bütün aşamalarda içindeki gereksinim maddelerine aykırı bir şey olmadığı, çalışmaların bu gereksinimleri sağladığı teyit edilir.
- **Tasarım aşaması:** Bu aşamada detaylı bir tasarım yapılır. Tasarımın her biriminin gereksinimlere uygunluğu kontrol edilir. Genelde herhangi bir kod yazılmaz. İleriki aşamalarda bu tasarıma sıkıca uyulur.
- **Gerçekleştirme aşaması:** Kodlama bu aşamada yapılır. Yazılan kodların birim testleri de bu aşamada yapılır. Yine her aşamada olduğu gibi gereksinimlerin sağlandığı kontrol edilir.
- **Doğrulama aşaması:** Bu aşamada detaylı test dokümanları yazılır ve bu dokümanlara göre testler yapılır. Her testin hangi gereksinimin gerçekleştiğini test ettiği izleme matrisi denilen tablolar ile test dokümanlarında bulunur. Modüller birbiri ile entegre edilir, karakutu ve saha testleri yapılır.
- **Bakım aşaması:** Bu aşamada sistem kullanıcının emrine verilmiştir ve bu sırada oluşacak olan problemler giderilir.

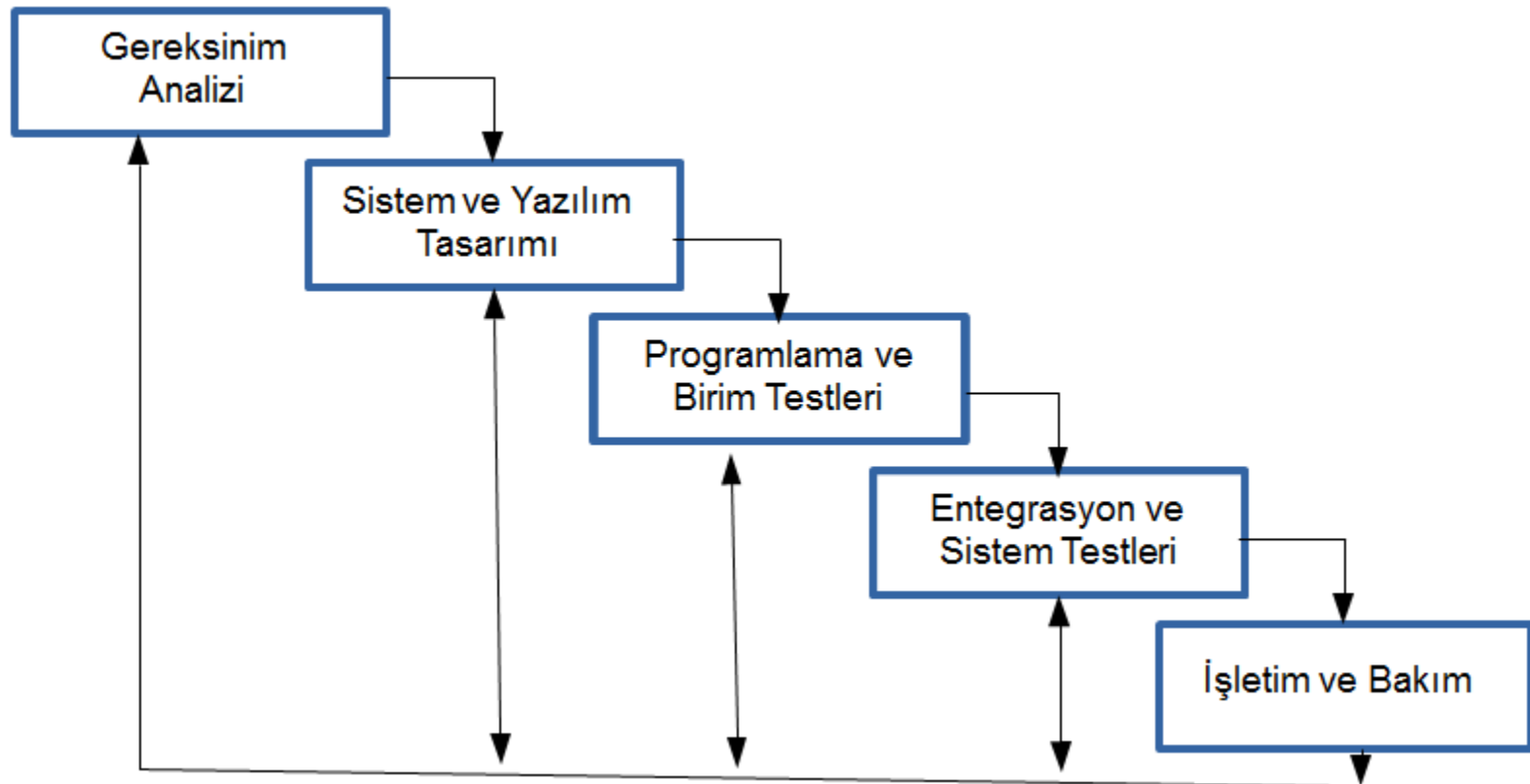
Yazılım Geliştirme Modelleri

Şelale (WaterFall) Yazılım Geliştirme Modeli (devam)



Yazılım Geliştirme Modelleri

Şelale (WaterFall) Yazılım Geliştirme Modeli (devam)



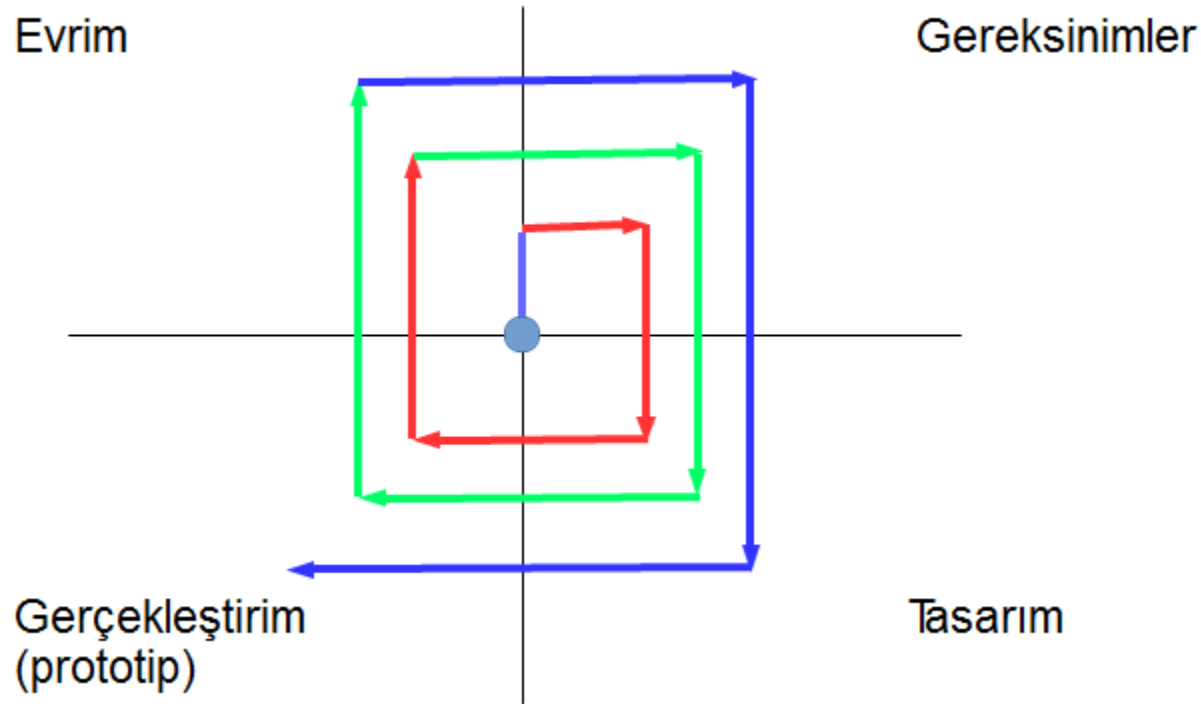
Yazılım Geliştirme Modelleri

🌀 Şelale Yönteminin Avantaj ve Dezavantajları

- Şelale yöntemi yazılım geliştirmede düzenliliği teşvik eden bir modeldir ve eskiden yazılım projelerinin kabusu olan kaos ortamının giderilmesinde büyük rol oynamıştır.
- Özellikle ciddi ve büyük projelerde düzen, dokümantasyon, detaylı tasarım yönlerinden kendini kanıtlamış bir modeldir.
- Ancak sonraları modelin sorunları ve dezavantajları olduğu keşfedilmiş ve yeni model arayışlarına gidilmiştir.
- Bu dezavantajlara değinecek olursak:
 - Bir safha tamamlanmadan diğer safhaya geçilemediğinden bir safhada problem olduğu ve o safha uzadığı zaman projelerin süresinin çok uzayabildiği görülür. Özellikle gereksinimlerin toplanması veya hazırlık safhasında bir hayli zaman kaybedilebilir.
 - Safhalarda daha önce yapılmış safhalarla geçiş kuralları (örneğin eksik kalan işler için geri dönmek gerekebilir) olmadığı için ya hiç geri bakılmaz ve önceki hatalar düzeltilemez ya da geriye dönülür ve birçok iş zincirleme olarak büyük emeklerle düzeltilir. Örneğin müşteri gereksinimi değiştiği zaman geri dönülmesi demek bu tüm döngünün yeniden geçilmesi demektir.
 - Oluşan hatalar tüm döngü boyunca her safhada tekrarlanacağı için özellikle gereksinimlerde oluşan hatanın sisteme maliyeti çok yüksek olur.
 - Tüm gereksinimlerin bir anda toplanması pratik değildir, ya bir şeyler eksik kalır ya da belirsiz eksik gereksinimler ile sistemin güvenilirliği azalır.
 - Müşteri geribeslemesi sadece proje başında yoğun olduğu için nihai ürünün müşterinin isteğini karşılamaması riski büyüktür.

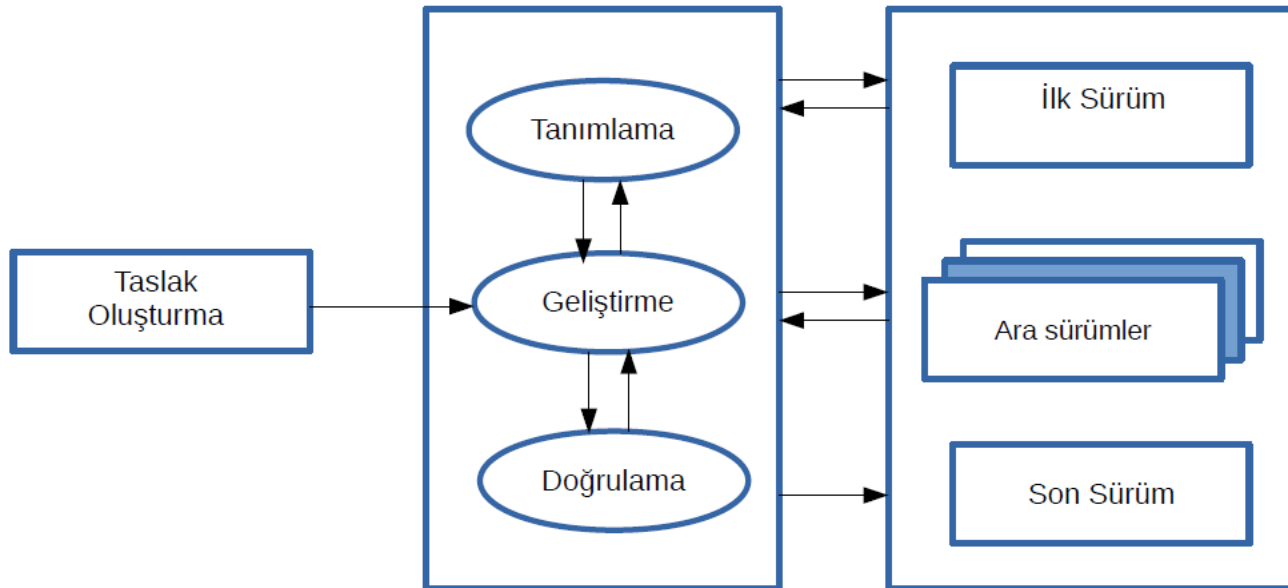
Yazılım Geliştirme Modelleri

Artırımsal (Incremental) ve Tekrar İşlemeli (İteratif) Yazılım Geliştirme Modeli



Yazılım Geliştirme Modelleri

Artırımsal (Incremental) ve Tekrar İşlemeli (İteratif) Yazılım Geliştirme Modeli



Yazılım Geliştirme Modelleri

Artırımsal (Incremental) ve Tekrar İşlemeli (İteratif) Yazılım Geliştirme Modeli

- Uzun ve kuralcı üretim safhaları olan Şelale (Waterfall) yönteminin dezavantajları görülünce buna alternatif arayışları ortaya çıktı.
- Bu yüzden artırımsal ve tekrar işlemeli yazılım geliştirme yöntem grubu ve bir takım alt sınıfları belirdi.
- Öncelikle geriye bir bakacak olursak Şelale yönteminin ana sıkıntısı şudur:
 - Şelale modelinde planlama, gerçekleştirim ve test safhaları sadece bir kez yapılır ve bu süre çok uzundur, sonuçta herhangi bir sorun çıktığında geri dönüp bunu telafi etmek mümkün olmaz veya maliyeti çok fazla olur.
 - Şelale yöntemi en başta tahmin ve planlamaya dayalıdır, bu yüzden proje başladıktan sonra değişiklik kararları almak çok zordur.

Yazılım Geliştirme Modelleri

Artırımsal (Incremental) ve Tekrar İşlemeli (İteratif) Yazılım Geliştirme Modeli (devam)

- Bu sorunu aşabilmek için temel olarak iki öneri yapılabilir:
 - Safhaları kısa tutarak bir kaç kez tekrar etmek ve her döngüde yeni özellikler eklemek (Artırımsal (Incremental) yazılım geliştirme)
 - Proje planlarını esnek, değiştirilebilir hazırlayıp proje esnasında sürekli iyileştirmelere imkân tanımak (Tekrar İşlemeli (İteratif) yazılım geliştirme)
- Artırımsal ve Tekrar İşlemeli Yazılım Geliştirme Modelinin birçok değişik yorumu ve uygulama örneği ya da diğer bir deyişle alt modelleri çıkmıştır.
- Bu alt modellerin her biri ana olarak bu iki kurala (yani Artırımsal Geliştirme ve Tekrar İşlemeli Geliştirme kurallarına) uymakla beraber kendi kurallarını ve çözüm önerilerini de getirmişlerdir. İlerleyen kısımlarda bu alt modelleri inceleyeceğiz.

"Acaba ne zaman artırımsal geliştirme yöntemini kullanmalı? Artırımsal geliştirme modelini sadece başarılı olmak istediğiniz projelerde kullanmalısınız."

Yazılım Geliştirme Modelleri

∞ Spiral Yazılım Geliştirme Modeli

- **Şelale** modeline en yakın fakat artırımsal olan yazılım geliştirme modeli Spiral yazılım geliştirme modelidir.
- Spiral yazılım geliştirme modeli “prototip yazılım geliştirme modeli” ile “şelale yazılım geliştirme modeli”nin her ikisinin de özelliklerini taşır.
- Şelale modelinden en önemli farkı bir tek döngü yerine birçok döngüden oluşmasıdır.
- Spiral model ilk olarak Barry Boehm tarafından 1988’de önerilmiştir.

Yazılım Geliştirme Modelleri

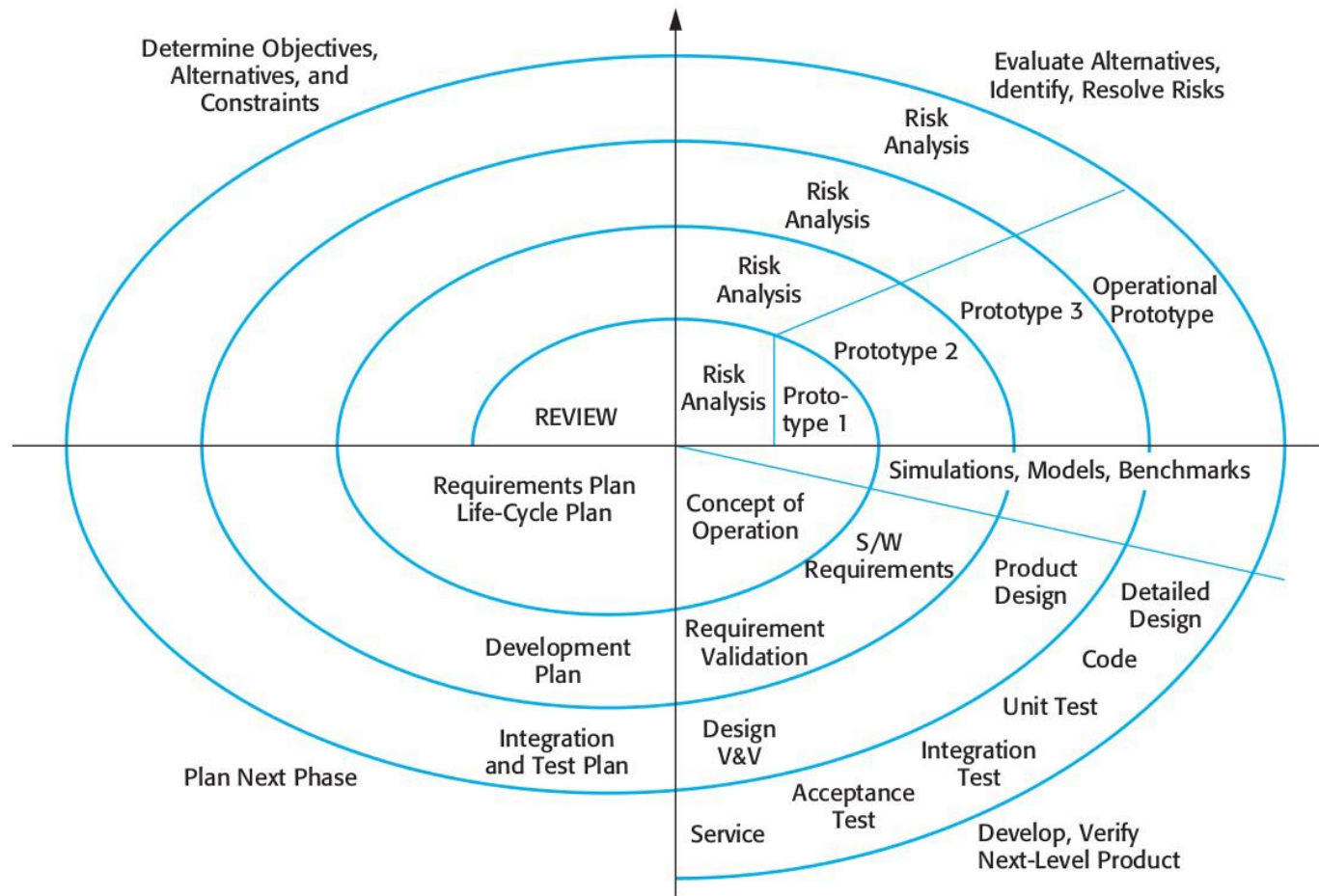
∞ Spiral Yazılım Geliştirme Modeli (devam)

- Spiral yazılım geliştirme modelindeki adımlar aşağıdaki gibi sıralanabilirler:
 1. Yeni üretilecek sistemin gereksinimleri mümkün olan en detaylı biçimde planlanır. Bu genellikle dış ve iç müşterilerin temsilcileri ve halihazırda bir sistem varsa onu inceleyerek yapılır.
 2. Yeni üretilecek sistem için bir ön tasarım hazırlanır.
 3. Ön tasarımdan faydalanılarak yeni sistem için ilk prototip hazırlanır. Bu genellikle daha az yetenekli ve nihai ürünün özelliklerini sadece kabaca taşıyan bir sistemdir.
 4. Daha sonra ikinci bir prototipin yapılması için şu işlemler yapılır:
 - a. İlk prototip kuvvetli ve zayıf yönlerine göre, risklerine göre, incelenir
 - b. İkinci prototip için gereksinimler belirlenir,
 - c. İkinci prototip planlanır ve tasarlanır,
 - d. İkinci prototip üretilir.
 5. Bu aşamada artık risk faktörleri rahatça belirlenebilir. Bütçe aşımalarının olup olmayacağı, işletim giderlerinin miktarı, vb faktörler hesaplanır ve müşteri bu risk faktörlerine göre devam etmeye veya projeyi sonlandırmaya karar verebilir.
 6. Üretilmiş olan prototip bir önceki prototip gibi incelenir ve gerekliyse bir başka prototipin daha yapımı için işe başlanır.
 7. Böylece ürün tatmin edici oluncaya kadar yeni prototiplere devam edilir, müşteri üründen memnun olunca bu prototip nihai ürün kabul edilir.
 8. Son prototipe göre nihai ürün üretilir.

Üretilen ürün detaylıca test edilir.

Yazılım Geliştirme Modelleri

Spiral Yazılım Geliştirme Modeli



Yazılım Geliştirme Modelleri

☞ Çevik (Agile) Yazılım Geliştirme Modeli

- Uzun ve kuralcı üretim safhaları olan Şelale (Waterfall) yönteminin dezavantajları görülünce buna alternatif arayışları ortaya çıktı.
- Karşılaştırma yapabilmek için öncelikle Şelale yönteminin dezavantajlarını kısaca hatırlayacak olursak:
 - Şelale modeli yoğun planlama gerektiren bir yöntemdir,
 - Planlamanın yoğun olması dokümantasyonun da yoğun olmasına yol açar,
 - Şelale modeli büyük ölçüde tahmin ağırlıklı bir yöntemdir,
 - Şelale modelinin safhaları uzun olduğu için ve kontratlar düzenlendiği için kolayca değişimle gidilemez.

Yazılım Geliştirme Modelleri

☞ Çevik (Agile) Yazılım Geliştirme Modeli (devam)

- Diyelim ki bu dezavantajları yenmek istiyoruz ama bu değişimin yan etkilerinden etkilenmek de istemiyoruz.
- Bu durumda yapılacak işlemler ve yan etkilerden kaçınma önlemleri şöyle olabilir:
 - Uzun vadeli planlamadan kurtulalım, ancak yan etkiden kaçınmak için:
 - Değişikliklere çabuk tepki verebilmeliyiz.
 - Ağır dokümantasyonu hafifletelim ancak bilgi kaybına uğramamak için:
 - Her safhada (tam fonksiyonel olmasa da) çalışan, sağlıklı bir kod bulundurmaliyiz,
 - Katı kontratlardan kurtulalım, ancak bu sırada zarara uğramamak için:
 - Müşterilerle sürekli iletişim içinde olmalıyız
 - Katı süreçlerden kurtulalım ancak bu arada bilgi akışım ve birikimini devam ettirebilmek için:
 - Geliştiriciler arası iletişimi maksimumda tutmalıyız.

Yazılım Geliştirme Modelleri

☞ Çevik (Agile) Yazılım Geliştirme Modeli (devam)

- İşte birlikte kolayca hazırladığımız yukarıdaki çıkarımlar aslında Çevik Yazılım Geliştirme Modelinin esaslarıdır. Bu esasları yazılım literatüründeki resmi söylemi ile aktarırsak:

“Biz yazılım geliştirmenin daha iyi yöntemlerini bizzat uygulayarak veya başkalarının uygulamasına yardımcı olarak gün yüzüne çıkarıyoruz. Bu çaba sonucunda bizim için:

- **Bireyler ve etkileşimler**, süreçlerden ve gereçlerden,
- **Çalışan yazılım**, kapsamlı dokümantasyondan,
- **Müşteri katılımı**, kontrat pazarlıklarından,
- **Değişime çabuk tepki verebilmek**, bir plan izlemekten daha değerlidir.

Şöyle ki, “Yukarıdaki maddelerin **sağ** tarafında yazılı olan öğeler değerlidir ancak sol taraftaki öğeler daha **çok değerlidir.**”

Yazılım Geliştirme Modelleri

🌀 Çevik (Agile) Yazılım Geliştirme Modeli (devam)

- Birçok sorunu çözmesine rağmen Çevik Programlamanın dezavantajları da vardır:
 - Yanlış uygulandığı zaman yani çeviklik kavramı yanlış anlaşıldığı zaman personelin tükenmesi (Employee burnout) dediğimiz yan etkiyi doğurur.
 - Ekip liderine çok fazla sorumluluk ve güç verildiğinde bu hem ekip liderini hem de ekip üyelerini yıpratır.
 - Çevik programlamanın her türlü yazılım problemini çözeceği düşünüldüğünde hayal kırıklıklarına yol açar.
 - Çevik programlama yazılım geliştiricilerin ve takımın disiplinli, düzenli ve tecrübeli olmasını tercih eder dolayısıyla buna hazır olmayan organizasyonlar, takımlar ve geliştiriciler için uygun değildir.
 - Dokümantasyon tamamen ihmal edilebilir ve bunun sonuçları kötü olabilir.
 - Büyük projelerde çok uygun değildir.

Yazılım Geliştirme Modelleri

∞ Uç (Extreme) Programlama Yazılım Geliştirme Modeli

- Artırımsal programlamanın birçok alt tipi veya uygulaması vardır.
- Bunlardan biri de Uç Programlama (Extreme Programming) yazılım geliştirme modelidir.
- Şelale yönteminin dezavantajlarından kaçınan bir yöntem olan Çevik (Agile) Programlamadan bahsetmiştik.
- Uç Programlama (Extreme Programming) bunu iyice öteye taşır.
- Şelale yönteminin dezavantajlarından büyük ölçüde kaçınır ve yan etkilere uğramamak için radikal önlemler alır.

Yazılım Geliştirme Modelleri

∞ Uç (Extreme) Programlama Yazılım Geliştirme Modeli (devam)

- Çiftler halinde (Pair) programlama: Hız sağlar, uzun vadeli planlama eksikliğini doğru tasarım ve kod ile kapatır,
- Kod gözden geçirme toplantıları: Uzun vadeli planlama eksikliğini doğru tasarım ve kod ile kapatır,
- Tüm kod için birim testler: Uzun vadeli planlama eksikliğini doğru kod ile kapatır,
- Gerçekten gerekmedikçe özellik eklememek: Sadelik sayesinde değişime ayak uydurmak için,
- Yatay yönetim şekli: Hız kazandırır,
- Kodda basitlik ve açıklık: Sadelik sayesinde değişime ayak uydurmak için,
- Müşteri değişikliklerine hoşgörü: Uzun vadeli planlama eksikliğini kapatır,
- Müşteri ile iyi iletişim: Uzun vadeli planlama eksikliğini kapatır,
- Programcılar arasında iyi iletişim: Hız kazandırır.

Yazılım Geliştirme Modelleri

∞ Uç (Extreme) Programlama Yazılım Geliştirme Modeli (devam)

- Uç programlamanın ana özelliklerini sıralayacak olursak:
 - Basitlik: Gerekli değişikliklere kolay ayak uydurabilmek için,
 - Geri besleme: Gerekli değişiklikleri zamanında edinebilmek için,
 - Cesaret: Kodda gerekli değişiklikleri yapabilmek için,
 - Saygı: Uyumlu çalışma ortamını sağlamak için.

Yazılım Geliştirme Modelleri

∞ Uç (Extreme) Programlama Yazılım Geliştirme Modeli (devam)

- Uç programlamanın eleştirilen yönleri de vardır:
 - Uç programlama ile 10 kişinin altındaki ekiplerde verimli sonuçlar elde edilirken büyük ekiplerde ve projelerde planlama eksikliğinden dolayı kötü sonuçlar verebilmektedir. Üstelik toplam kalite kurallarına göre mutlaka hazırlanması gereken dokümanlar “Biz Uç Programlama yapıyoruz” mazereti altında yazılmamakta; ihmaller veya suistimaller olabilmektedir.
 - Uzun vadeli planlama yapılmadığı için hedef sapmaları, koda eklenmiş gereksiz özellikler rahatlıkla oluşabilmektedir,
 - Uç Programlamanın sürekli iletişim ve hatta sözlü iletişim gerektiren çalışma biçimine (çok kaliteli yazılımcılar olsalar bile) herkes ayak uyduramamaktadır. Üstelik ekipteki herkesin iyi anlaşamaması da olası bir durumdur,
 - Sadece kısa vadeli planların olduğu bu ortamda gerçekten tecrübeli programcılar gerekmektedir,
 - Müşteri ile kontrat sorunları ve çıkan ürün özellikleri üzerinde sonradan sorunlar çıkabilmektedir,
 - Ana özelliklerden olan müşteri katılımı her zaman mümkün olmamaktadır,
 - Uzun vadeli detaylı planlama olmadığı için proje tahminleri zorlaşmaktadır,
 - Uç Programlama özellik ekleme temelli olduğu için görünür özellikler arasında olmayan ama altyapı için gerekli olan önemli bazı proje yetenekleri kaçırılabilir (Örneğin gömülü bir projeye alelacele makine dili ile başlayıp zaman geçtikçe aslında işletim sistemi ve yüksek seviyeli bir dil gerektiği gerçeği ile karşılaşmak gibi.)

Yazılım Geliştirme Modelleri

🌀 Uç (Extreme) Programlama Yazılım Geliştirme Modeli (devam)

- Uç Programlamanın en belirgin özelliği kısa iterasyonlara sahip olması ve hemen her zaman çalışan bir sistemin (basit ve özelliksiz bile olsa) paydaşlara sunulabilmesidir.
- Bu şekilde bir çalıştırılabilir programın olması, projenin gidişatının sürekli kontrol altında tutulabilmesini sağlar. Oysa klasik yöntem olan şelale modelinde, son ana kadar başarılı görünen bir yazılım, son inşa sonrasında hiç çalışmayabilir bile.

"Çalışan bir karmaşık sistem kaçınılmaz olarak çalışan basit bir sistemden geliştirilmiştir. Tam ters ilişki de geçerli gibi görünüyor: Çöplük gibi bir sistemden geliştirilen bir sistem de asla çalışmaz ve çalıştırılması becerilemez. En baştan çalışan bir basit sistem kurarak işe başlamak zorundasınızdır."

Gali kanunu (Galls law)- John Gali

- Uç programlamanın bir diğer özelliği de YAGNI prensibidir. YAGNI "You ain't gonna need it" yani Türkçe karşılığı ile "İhtiyacın olmayacak ki" cümlesinin sözcüklerinin baş harflerinden oluşur. Felsefi olarak bir özelliğin ancak gerçekten ihtiyaç çıktığında koda eklenmesini önerir. İhtiyaç yokken belki ileride ihtiyaç olur diye boşu boşuna kodu karmaşıklaştırmaya ve büyütmeye karşıdır.

Yazılım Geliştirme Modelleri

∞ Spiral ve Çevik Yazılım Geliştirme Modelleri Arasındaki Farklar

∞ Spiral ve çevik yazılım geliştirme modelleri birbirleri ile karıştırılabilirler. Her ikisi de artırimsal ve tekrar işlemeli yazılım geliştirme modelinin alt üyesidirler. Birbirlerine benzemekle beraber aralarında bazı farklar vardır:

- Spiral modelde ön planlama daha ağırlıklıdır, iterasyonlar önceden planlanır. Oysa Çevik yazılım geliştirmede sadece o anki iterasyon için detaylı plan yapılır, diğer iterasyonlar için plan yapılmaz, belirsizlik zaten kabul edilen bir gerçektir.
- Spiral modelde “planla - gerçekleştir(kodla) - test et - düzelt” tipi bir yaklaşım varken Çevik modelde testlerin mümkün olan en kısa zamanda hatta bazen iterasyonun en başında hazırlanması düşünülür. Çevik modelde test kodlamadan daha önceliklidir.
- Çevik modelde her iterasyon sonunda, öncelikli gereksinimlerin gerçekleştirildiği, çalışabilir, tamamen test edilmiş, az özellikli ama hazır bir program üretilir. Spiral Modelde bu şart değildir, ilk iterasyonlarda sadece bir prototip mahiyetindeki bir ürün ortaya çıkabilir.
- Spiral model teknik bir üretkenlik yaklaşımı iken buna ek olarak Çevik Model insan psikolojisini de göz önüne alan, sosyal içerikleri olan bir geliştirme modelidir.
- Spiral model daha klasik bir yaklaşım olduğu için dokümantasyondan, kod içi yorumlardan, planlardan ve gereksinimlerden taviz vermez, Çevik modelde ise bu konular suistimale biraz daha açıktır.

Yazılım Geliştirme Modelleri

Scrum Yazılım Geliştirme Modeli

- Scrum küçük takımlar için bir yazılım geliştirme modelidir. Scrum yazılım geliştirme modelinde takım elemanlarının sayısı 10'u geçmemelidir. Birden fazla takım varsa bunların birbirinden bağımsız çalışabilmesi şartları hazırlanmalıdır.
- Scrum, Amerikan futbolunda yeni hamleyi yapmak için bir arada, kenetlenmiş olan takıma ve yaptıkları hamleye denir, bu yazılım yöntemi ve yöntemi kullanan yazılım ekibi de buna benzetilmiştir.
- Projede ilk safha planlama safhasıdır. Bu safhada bir ürün mimarisi ve planı geliştirilir ve şef yazılım mimarı seçilir. Projenin yapısından şef yazılım mimarı sorumludur. Proje sırasında yazılım mimarisi değişebilir ancak değişsin veya değişmesin mimariye proje boyunca uyulmaya çalışılır.
- Planlama safhasından sonra bir seri kısa geliştirme safhalarından geçilir. Her safha “depar” (sprint) olarak adlandırılır ve projenin artırımsal olarak ilerlemesini sağlar. Her bir safha (depar, sprint) genelde 1 ila 4 hafta arasında sürer. Kapanış safhası ile proje sonlandırılır.

Yazılım Geliştirme Modelleri

Scrum Yazılım Geliştirme Modeli (devam)

- Yazılım takımı bütün görevleri “işlem maddesi” (backlog) olarak adlandırılan bir listede tutar. Bu “işlem maddeleri” takımın ana güdüleyicisidir. Her depar safhasından önce bu “işlem maddeleri” güncellenir, öncelikleri ayarlanır ve yazılım takımı bu maddelerden gözüne kestirdiklerini yapmak için onay verir. Takımın her üyesi kendisine düşen birkaç maddeyi depar sırasında tamamlamak için onay verir.
- Genelde bu depar süreleri hafta(lar) boyutunda olmalıdırlar. Büyük görevlerde ve süre uzadıkça görevi tanımlamak ve iş sırasında rapor vermek güçleşmektedir. Bir depar sırasında takım dışından kaynaklı bir değişikliğin takıma rica edilmesine izin verilmez.
- Her depar (geliştirim safhası, sprint) sırasında ürüne gözlenebilir, kullanılabilir veya dışarı sunulabilir bir artım yapılmaya çalışılır. Her deparın ardında yatan felsefe değerli bir işlevi ürüne katmaktır.
- Deparlar sırasında hedefin depar süresince bitirilmesine çalışılır. Depar süresinde hedef bitmez ise deparın süresi asla değiştirilmez, bunun yerine gerekliyse hedef yeniden tanımlanır veya küçültülür. Depar tarihleri ile oynamamak takım içinde genel bir standardı sağlar.
- Depar sırasında, sıkça, takımın her üyesinin katıldığı toplantılar yapılır. Bunlar genelde günlük toplantılar olarak organize edilir. Böylece projedeki olası kaymalar, gecikmeler çok kısa zamanda tespit edilebilir. Toplantılara uzakta çalışan üyeler telekonferansla bile olsa çağrılır, böylece takım ruhu ayakta tutulur.

Yazılım Geliştirme Modelleri

Scrum Yazılım Geliştirme Modeli (devam)

- Scrum modelinde müşteri ile irtibatlı olan pazarlama birimi ile teknik geliştirme birimi arasında sıkı iletişim ve birlikte çalışma olmalıdır. Tahminlerin yapılmasında ve hedeflerin tutturulmasında bu çok önemlidir. Proje sırasında hedeflerin öncelikleri bu iki ekip tarafından ayarlanarak proje için hayati özelliklerin üründe bulunmasına çalışılır.
- Scrum şefi (şef yazılım mimarı) işlem maddelerini (backlog) hazırlar, günceller ve depardan önce kullanılabilir duruma getirir. Ayrıca projedeki ilerlemeyi ölçer, işlem maddelerinin tamamlandığını takip eder, riskleri hesaplar ve acil önlemler alır. Serum toplantılarının kısa ve amaca odaklı olmasını kontrol etmek de Scrum şefinin görevidir.
- Her scrum toplantısı 15-30 dakika arasında sürer. Toplantıda tamamlanan işlem maddeleri, karşılaşılan engeller ve bunları çözmek için yapılan planlardan söz edilir. Sorunların çözüm yöntemleri için hararetli tartışmalara girilmez ve bunlar için sadece ilgili taraflarla başka toplantılar yapılır.
- Her depar sonunda tüm deparın sonucunu gözden geçirilir ve her konuda değişiklik yapılabilir. En son depar ürününün teslimi için yapılır.

Yazılım Geliştirme Modelleri

∞ Spiral ve Scrum Yazılım Geliştirme Modelleri Arasındaki Farklar

- Scrum yazılım geliştirme modelinin bileşenleri ve fikirleri aslında yeni değildir.
- Scrum modeli Spiral Yazılım Geliştirme Modeline benzer. Basitçe, zaman adımlarına bölünmüş, tipik bir artırımsal yazılım geliştirme modelidir.
- Klasik artırımsal modelden fazla olarak “işlem maddelerinin durumunun, karşılaşılan engellerin ve bu engelleri aşmak için yapılan planların konuşulduğu sık toplantılar” vardır.
- Bunun dışında Spiral modelin biraz daha hızlandırılmışı olarak kabul edilebilir.
- Scrum toplamda bir kaç ay süren, her deparı (geliştirme safhası) birkaç hafta süren, gereksinimleri başlangıçta tam belirli olmayan projeler için uygundur.

Yazılım Geliştirme Modelleri

∞ Birleşik İşlem (Unified Process) Yazılım Geliştirme Modeli

- Bir diğer artırımsal (ineremental) ve tekrar işlemeli (iterative) yazılım geliştirme modeli de Birleşik İşlem yazılım geliştirme modelidir.
- Modelin en büyük özelliği İşlev ve kullanım öykülerini (Use Case) temel öge olarak almasıdır.
- Mimari temelli bir modeldir. Risklerin en kısa zamanda belirlenmesi ve en büyük riske öncelik verilmesine önem verir.
- Dört tane ana safhası vardır:
 - Yakalama safhası (Inception),
 - Ayrıntılandırma safhası (Elaboration),
 - Üretim safhası (Construction),
 - Geçiş safhası (Transition).

Yazılım Geliştirme Modelleri

🌀 Birleşik İşlem (Unified Process) Yazılım Geliştirme Modeli

- **Yakalama safhası** projenin ön hazırlığıdır ve mümkün olduğunca kısa olması istenir. Projenin hedefi, sınırları, riskleri, kullanıcı ve işlev öyküleri, ana gereksinimleri bu safhada belirlenir. Kaba bir proje planı ve maliyet tahmini yapılır. Proje için uygun aday mimariler belirlenir.
- **Ayrıntılandırma safhası**nda sistem gereksinimleri detaylandırılır. Risk faktörleri hesaplanır ve sistem mimarisi hazırlanır ve doğrulanır. Doğrulama için projenin ana çatısı gerçekleştirilebilir. Türlü şemalarla detaylar desteklenir.
- **Üretim safhası** en kapsamlı safhadır ve bu sırada ürünün detaylı üretimi gerçekleştirilir. Üretim kısa zaman adımlarından oluşan yinelemelerle gerçekleştirilir.
- **Geçiş safhası** ürünün sahaya sürülmesi, kurulması veya müşteri tarafına sağlıklı biçimde iletilmesi safhasıdır. Burada ürün sonlandırılana kadar çeşitli iyileştirme çalışmaları yapılabilir.

Yazılım Geliştirme Modelleri

∞ Yalın Yazılım Geliştirme Modeli

- Yalın Yazılım Geliştirme modeli (Lean Software Development) endüstrideki verimli kaynak kullanımı modelini yazılım projelerine de uygulamayı hedefler (Hibbs 2008).
- Endüstride kullanılan verimli kaynak modeli esaslarını listeleyecek olursak:
 - Gereksiz stoklama yoktur,
 - Mümkün olduğunca otomasyon yapılır,
 - İsraf yoktur, kaçınılacak israf maddeleri şunlardır:
 - Hatalar: Klasik hatayı bul ve tamir et yöntemi yerine önleyici tedbirlerle hatalar oluşmadan engellenmelidir,
 - Aşırı üretim; Kaçınılması gereken diğer bir israftır,
 - Taşıma: Parçaların işlemler arasında taşıma maliyetleri hesap edilmelidir,
 - Bekleme: Parçalar veya insanların beklemesi maliyeti arttırır,
 - Stok: Stok para akışını bozacağı için, ödeme dengesini bozar, maliyeti arttırır, ayrıca alan ve çalışan maliyetini arttırır,
 - Hareket: gereksiz yere ekipman veya insanların hareketi de zaman kaybettirir,
 - Gereksiz işleme: Müşterinin istediğinden daha fazla işleme yapmak maliyetleri arttırır,
 - İnsanların faydasız kullanımı: Maliyetleri arttırır.

Yazılım Geliştirme Modelleri

∞ Yalın Yazılım Geliştirme Modeli (devam)

- Yukarıdaki verimlilik temelli modeli yazılım geliştirmeye de uygulamayı amaçlayan Yalın Yazılım Geliştirme modeli aynı maddeleri şu şekilde yazılım için yorumlar:
 - **Tam zamanında işlem ve gereksiz stoklamadan kaçınmak:** Bitmemiş, yarım durumda her yazılım maliyeti arttırır bu sebeple yarım işlerden kaçınılır,
 - **Otomasyon:** Teknolojinin sağladığı tüm gereçlerden faydalanılır, bu bağlamda otomatik kod yazma, versiyon tutma, hata raporlama, tasarım programları yoğun biçimde kullanılır.
 - **İsraftan kaçınmak:** Kaçınılacak israf maddeleri şunlardır:
 - **Hatalar:** Tüm aşamalarda kalite içeri gömülür, gereksinimlerin hazırlanmasında, tasarımın yapılmasında, vb hatalar minimuma indirilir ve böylece ürün çıktıktan sonra oluşacak hatalar azaltılır, yani kalite tüm işlem süresine gömülür ve dağıtılır,
 - **Aşırı üretim, ekstra özelliklerden kaçınılmalıdır:** Kullanıcının isteklerinin %80'inin ürünün özelliklerinin %20'si ile karşılanabildiği göz önüne alınmalıdır,
 - **Taşıma:** Bilgiler, dokümanlar toplantı raporları herkese aynı anda iletilmelidir, müşteriden gelen bir gereksinim planlama bölümünden tasarımcıya ulaşana kadar defalarca el değiştirmekte ve hatalarla dolmaktadır, kulaktan kulağa şeklindeki iletişim biçimlerinden kaçınılmalıdır,
 - **Bekleme:** Projeler çok iyi planlamak ve beklemeler en aza indirilmelidir,
 - **Stok:** Yarım veya sürmekte olan birçok proje stok maliyetinin artması demektir, projeler kısa ara hedeflere sahip olmalı ve sürüncemede bırakılmamalıdır,
 - **Hareket:** Yazılımcılar sadece bir işle uğraşmalı birden fazla proje bir yazılımcıya atanmamalıdır, bir anda bir projeden diğer bir projeye adapte olma süresi çok fazladır,
 - **Gereksiz işleme:** Gereksiz süreçler bir an önce elimine edilmelidir.

Yazılım Geliştirme Modelleri

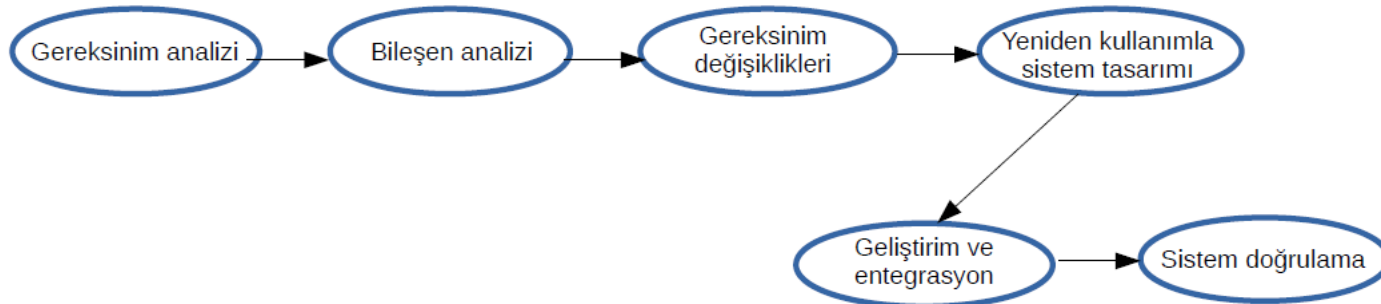
🌀 Yalın Yazılım Geliştirme Modeli (devam)

- Yalın programlamanın bu esaslardan çıkardığı bazı öneriler de vardır:
 - **İnsanlara saygı gösterin:** Yalın programlamanın esası kaynakları iyi kullanmaktır demiştik. İnsanlar da en önemli kaynaklar olduğuna göre insanlara saygı göstermek, kendilerini yetiştirmelerine gelişmelerine destek olmak, fikirlerine değer vermek ve iyi bir çalışma ortamı sağlamak Yalın Programlamanın temel hedeflerindendir. İnsanlara değer vermek, insanların verimlilik kuralları içerisinde aşırı yıpranmasını önler, yani katı verimlilik sistemini biraz daha insanileştirir,
 - **Araçlar kullanın:** Versiyon kontrol araçları, otomatik script'ler kullanın,
 - **Otomatik test imkânlarını kullanın:** Testlerinizi otomatikleştirin,
 - **Sürekli entegrasyon:** Sürekli birleştirme yaparak süreci iyileştirin, birleştirmeyi en sonda yapmayın ve sürece yayın,
 - **Daha az kod yazın:** Kodda sürekli iyileştirme (refactoring) yapın ve kaliteli fakat az kod yazın, daha çok kod daha çok problem demektir, yazılan her kod satırı harcanan para demektir ve bu sebeple asla gereksiz kod yazılmamalıdır. Müşteri gereksinimleri çok iyi irdelenmeli ve gerçekten gerekli olmayan gereksinim çıkarılmalı ve dolaylı olarak gereksiz kodun yazılması engellenmiş olmalıdır.
 - **Kısa iterasyonlar yapın:** Böylece süreci daha bir kontrol altında tutun,
 - **Müşteri katılımı:** En baştan en sona kadar müşteri katılımını sağlayın.

Yazılım Geliştirme Modelleri

Re-use (Tekrar Kullanımlı) Yazılım Geliştirme Modeli

- Bu modelde sistem ve organizasyonun çoğu ürünü tekrar kullanılabilir modüllerden faydalanılarak üretilir.



Prototip Yazılım Geliştirme Modeli

- Bu modelde üretilecek modelin bir prototipi yapılır.
- Bu prototip sadece sistemi genel olarak görmek için yapılan hızlı ve özensiz geliştirilmiş bir örnek olduğu için genelde bu prototipin gerçek sistemin kalitesini etkilememesi için prototip kullanılmaz ve atılır.

"En kısa zamanda bir prototip yapınız."

Mike Gancarz ve arkadaşları, UNIX felsefesi maddelerinden

Yazılım Geliştirme Modelleri

∞ Test GÜdümlü Yazılım Geliştirme Modeli

- Bu yazılım modelinde öncelikle testler yazılır daha sonra bu testleri geçebilecek özelliklere sahip kod yazılır.
- Genellikle otomatik testler ile çalışmak süreyi çok kısaltır.
- Bu modelin detaylarına inecek olursak:
 - İlgili gereksinim için test kodunu yaz,
 - Testin geçemediğini teyit et (henüz herhangi bir işlev geliştirilmediğinden dolayı),
 - İlgili özellik için kod yaz,
 - Kodu test et ve geçtiğini göster,
 - Gerekliyse kodda iyileştirme ve ayarlamalar yap,
 - Kodu test et ve geçtiğini göster,
 - Başa dön.

Yazılım Geliştirme Modelleri

∞ Test GÜdümlü Yazılım Geliştirme Modeli (devam)

- Test GÜdümlü yazılım geliştirme modelinin uygulanmasının zor olacağı alanlar da vardır. Örneğin:
 - Grafikselsel kullanıcı arabirimi testleri genellikle otomatik testler için uygun değildir,
 - Testleri yazmak bazen hedef koda kıyasla çok zaman alıcı olabilir,
 - Test kodunun da bakımını yapmak gerektiğinden bakım maliyeti artar,
 - Test kodunun sürekli güncel tutulması gereklidir ve bu zor olabilir,
 - Testi yazan kişi ile hedef kodu yazan kişi aynı geliştirici ise hataları tespit edemeyebilir,
 - Test yazmak zaman kaybı olarak görülebileceğinden bu metoda karşı psikolojik direnç gösterenler olabilir.

Yazılım Geliştirme Modelleri

☞ Temiz Oda Yazılım Geliştirme Modeli

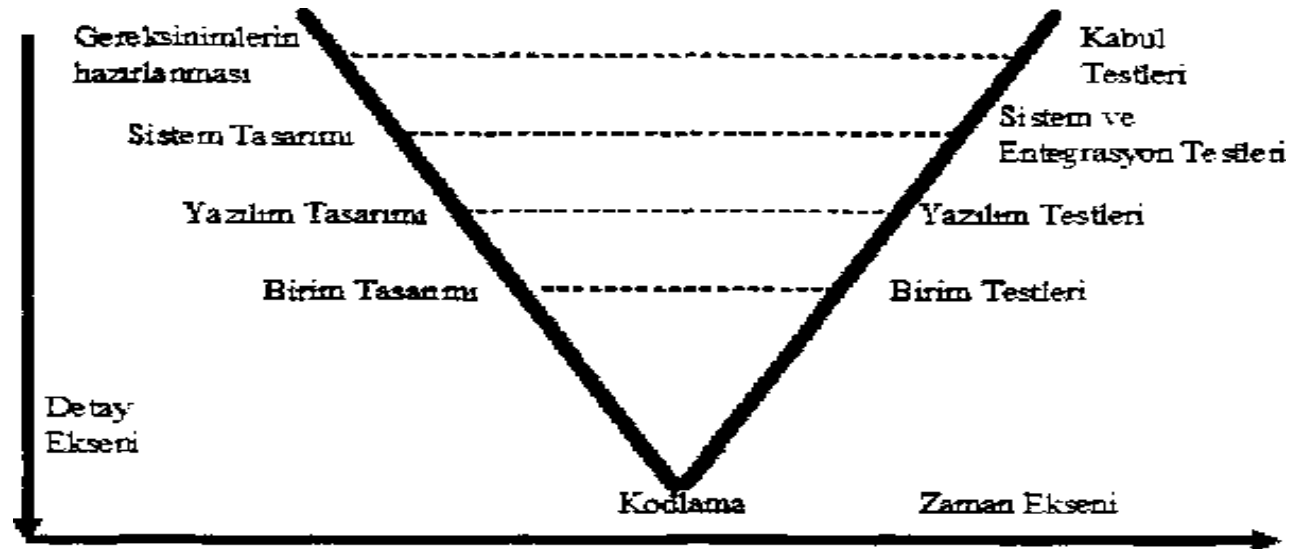
- Bu model son teknoloji yongaların üretildiği temiz odalardan esinlenerek seçildiği için temiz oda modeli adını almıştır.
- Bu model kovboy tarzı kuralsız programlama tipinin neredeyse tam tersidir. Modelin amacı neredeyse sıfır hata ile yazılım üretmektir.
- Bu amaca ulaşmak için:
 - Ön-tasarımda bir hayli vakit harcanır,
 - Kaliteyi sağlamak için istatistik! yöntemler yoğun olarak kullanılır,
 - Gereksinimler çok net ve açık belirlenir.
- Bu modelde üründe hata çıkması ve düzeltilmesi (Error Correction) beklenmez.
- Modelin ana fikri hatanın oluşmasını engellemektir (Error Prevention). Kalite kaygısı düşünülerek harcanan çabalar bu modeli destekler.
- Temiz oda yazılım geliştirme modeli Şelale Modeli kadar kuralcı olsa da yöntem olarak artırımsal (iteratif) yaklaşım benimsenmiştir, böylece kalite her adımda denetlenebilmektedir.

Yazılım Geliştirme Modelleri

∞ V-Modeli

- Modelin grafik gösterimi V harfine benzediği için model bu isimle anılır.
- Grafikte sol taraf tasarım, sağ taraf ise doğrulama aktivitelerini içerir. Modelin ana ilkeleri şunlardır:

- Her işlem için karşılık gelen bir doğrulama aktivitesi olmalıdır.
- Geliştirme işlemi, gereksinimler sayesinde, ana hatları ile başlar, detaylara inilir, daha sonra bu detaylar doğrulanarak birleştirilir ve sonuçta ürün elde edilir.



Şekil 8: V-Modeli: Test merkezli bir geliştirme modeli

Yazılım Geliştirme Modelleri

∞ V-Modeli (devam)

- V-modeli geliştirme işlemine çok kabaca baktığı için, yinelemeli (iteratif) bir yaklaşım sunmadığı için ve pratikte kullanılabilecek çok fazla yöntem önermediği için eleştirilen bir modeldir.
- Bu model, belirsizlikleri az olan, yani iteratif yaklaşım gerektirmeyen, çok düzenli projelerde kullanılabilir.
- Modelin bir avantajı test edilmemiş herhangi bir aşama bırakmamasıdır.
- Diğer bir avantajı da iş akışında eksik kalan, atlanan bir aşama bırakmamasıdır.

Yazılım Kalite Unsurları

Yazılım Kalite Unsurları

- Kalitenin birçok tanımı yapılmıştır. Bunlardan birkaçını sıralarsak:
 - Kalite amaca uygunluktur.
 - Kalite insanlar için değer taşımaktır.
- Yazılım kalitesini belirleyen birçok özellik vardır.
- Yazılım hemen hemen tamamen hatalarından arındırılmış olsa bile kaliteli olmayabilir.
- Yani hatasızlık bir kalite ölçütüdür ancak kaliteyi belirleyen yegane özellik değildir.
- Üstelik bu özelliklerden bazıları bir ürün için daha önemli iken bazıları diğer bir ürün için daha önemli olabilir.

Yazılım Kalite Unsurları

Yazılım Kalite Unsurları

- Yazılım kalitesini belirleyen özellikler büyük ölçüde McCall (1977) ve Boehm'in (1978) çalışmaları ile belirlendi.
- Bu bilgilerden de faydalanarak yazılım kalitesini belirleyen unsurları şöyle sıralayabiliriz:
 - **Anlaşılabilirlik (Understandability):** Gerek sistem yazılımı, gerekse dokümanlar anlaşılır ve açık mı?
 - **Tamlik (Completeness):** Tüm gerekli bileşenler gerçekleşmiş mi? Sistem kaynakları yeterli mi?
 - **Doğruluk (Correctness):** Sistem kendisinden beklenen işlevleri doğru olarak yerine getiriyor mu?
 - **Taşınabilirlik (Portability):** Yazılım başka bir platforma taşınabiliyor mu? Taşınamıyorsa bu belirtilmiş mi?
 - **Sürdürülebilirlik-Bakım yapılabilirlik (Maintainability):** Yazılımın ileride çıkabilecek sorunları kolayca çözülebiliyor mu?
 - **Test edilebilirlik (Testability):** Yazılım kolayca ve doğru olarak test edilebiliyor mu?
 - **Kullanıma uygunluk (Usability):** Yazılım kullanıcı dostu mu?
 - **Güvenilirlik (Reliability) ve Sağlamlık (Robustness):** Yazılım kendisinden beklenenleri gerek süre gerekse koşullara bağlı olmaksızın doğru olarak yerine getirebiliyor mu?
 - **Etkinlik (Efficiency):** Yazılım (bellek, mikroişlemci kullanımı, haberleşme ağ kullanımı gibi konularda) hem etkin biçimde yazılmış hem de etkin bir biçimde çalışabiliyor mu?

Yazılım Kalite Unsurları

☞ Yazılım Kalite Unsurları (devam)

- **Güvenlik (Security):** Yazılıma sadece yetkisi olan kullanıcılar erişebiliyor mu? Yazılımın kendisi ve verileri yetkisiz erişime karşı korunuyor mu?
- **Esneklik (Flexibility):** Yazılıma gerektiği zaman yeni modüller veya işlevler kolayca eklenebiliyor mu?
- **Tekrar kullanılabilirlik (Reusability):** Yazılımın modülleri veya tamamını başka projelerde kullanmak mümkün mü?
- **Tutarlılık (Consistency):** Yazılımda kullandanan değişken adları, matematiksel veya çeşitli semboller ve ekrandaki elemanlar tutarlı ve birbiri ile uyumlu mu?
- **Farkındalık (Conciseness):** İlgili ürün yazılım kuralları, standartları gözetilerek, yeniden düzenleme ve optimizasyon kurallarına uyularak ve farkında olunarak yazılmış mı?
- **Erişilebilirlik-Edinilebilirlik (Availability):** Yazılıma piyasada veya herhangi bir kanalla erişmek kolay mı?
- **Kurulabilirlik (Installability):** Yazılımı kurmak kolay mı?
- **Uyumluluk (Compatibility):** Yazılım diğer sistemlerle uyumlu olarak çalışabiliyor mu?

"Bir programı güvenli, güvenilir ve hızlı yapmanın tek yöntemi onu küçük yapmaktır."

Andrew S. Tanenbaum